Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: L15+L16
Regno: *23BCE1344*	Name: *Aditya*

EX2A

S-DES CRYPTOGRAPHIC ALGORITHM

Index	
Code (Sender, Receiver)	
Output (Sender, Receiver)	

File name: Ex2a _23BCE1344.pdf

Code: (--CyptoFunctions--)

Cypto_functions.py:

```
# crypto_functions.py
P10 = [3,5,2,7,4,10,1,9,8,6]
P8  = [6,3,7,4,8,5,10,9]
IP  = [2,6,3,1,4,8,5,7]
IP_INV = [4,1,3,5,7,2,8,6]
EP  = [4,1,2,3,2,3,4,1]
P4  = [2,4,3,1]
S0 = [
    [1,0,3,2],
    [3,2,1,0],
    [0,2,1,3],
    [3,1,3,2]
]
S1 = [
    [0,1,2,3],
    [2,0,1,3],
    [3,0,1,0],
    [2,1,0,3]
]
def permute(bits, table):
    return ''.join(bits[i-1] for i in table)
def left_shift(bits, n):
    return bits[n:] + bits[:n]
def xor(a, b):
    return ''.join('0' if i == j else '1' for i, j in zip(a, b))
def sbox(bits, box):
    row = int(bits[0] + bits[3], 2)
    col = int(bits[1] + bits[2], 2)
    return format(box[row][col], '02b')
def generate_keys(key):
    p10 = permute(key, P10)
    l, r = p10[:5], p10[5:]
    l1 = left_shift(l, 1)
    r1 = left_shift(r, 1)
    k1 = permute(l1 + r1, P8)
    l2 = left_shift(l1, 2)
    r2 = left_shift(r1, 2)
    k2 = permute(l2 + r2, P8)
    return k1, k2
```

```python
def fk(bits, key):
    l, r = bits[:4], bits[4:]
    ep = permute(r, EP)
    x = xor(ep, key)
    s0_out = sbox(x[:4], S0)
    s1_out = sbox(x[4:], S1)
    p4 = permute(s0_out + s1_out, P4)
    return xor(l, p4) + r
# ENCRYPTION
def sdes_encrypt(pt, key):
    print("\n--- ENCRYPTION ---")
    print("Plaintext :", pt)
    k1, k2 = generate_keys(key)
    print("Subkey K1 :", k1)
    print("Subkey K2 :", k2)
    ip = permute(pt, IP)
    print("After IP  :", ip)
    r1 = fk(ip, k1)
    print("Round 1   :", r1)
    swapped = r1[4:] + r1[:4]
    r2 = fk(swapped, k2)
    print("Round 2   :", r2)
    ct = permute(r2, IP_INV)
    print("Ciphertext:", ct)
    return ct
# DECRYPTION
def sdes_decrypt(ct, key):
    print("\n--- DECRYPTION ---")
    print("Ciphertext:", ct)
    k1, k2 = generate_keys(key)
    print("Subkey K1 :", k1)
    print("Subkey K2 :", k2)
    ip = permute(ct, IP)
    print("After IP  :", ip)
    r1 = fk(ip, k2)
    print("Round 1   :", r1)
    swapped = r1[4:] + r1[:4]
    r2 = fk(swapped, k1)
    print("Round 2   :", r2)
    pt = permute(r2, IP_INV)
    print("Plaintext :", pt)

    return pt
```

Sender (server.py):

```
import socket
import crypto_functions as cf

key = "1100011110"
def encrypt(text):
   return cf.sdes_encrypt(text, key)
def decrypt(text):
   return cf.sdes_decrypt(text, key)

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = "127.0.0.1"
port = 12345
server_socket.bind((host, port))
server_socket.listen(1)
print(f"Server listening on {host}:{port}")

conn, addr = server_socket.accept()
print("Connected to", addr)
while True:
   plaintext = input("\nEnter 8-bit plaintext (or exit): ")
   if plaintext.lower() == "exit":
       conn.send(plaintext.encode())
       break
   cipher = encrypt(plaintext)
   conn.send(cipher.encode())
conn.close()
server_socket.close()
```

Receiver (client.py):

```python
import socket
import crypto_functions as cf

key = "1100011110"
def encrypt(text):
    return cf.sdes_encrypt(text, key)

def decrypt(text):
    return cf.sdes_decrypt(text, key)

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
host = "127.0.0.1"
port = 12345
client_socket.connect((host, port))
print("Connected to server")
while True:
    cipher = client_socket.recv(1024).decode()
    if cipher.lower() == "exit":
        break
    decrypt(cipher)
client_socket.close()
```

Output:

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex2a$ python ./server.py

Enter 8-bit plaintext (or exit): 00101000

--- ENCRYPTION ---
Plaintext : 00101000
Subkey K1 : 11101001
Subkey K2 : 10100111
After IP  : 00100010
Round 1   : 00110010
Round 2   : 00010011
Ciphertext: 10001010

Enter 8-bit plaintext (or exit):
```

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex2a$ python client.py
Connected to server

--- DECRYPTION ---
Ciphertext: 10001010
Subkey K1 : 11101001
Subkey K2 : 10100111
After IP  : 00010011
Round 1   : 00100011
Round 2   : 00100010
Plaintext : 00101000
```